COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 6, 2026	**DATE SUBMITTED:**	Oct 6, 2026
NAME:	Alvarez, Joshua Caleb P. Vince Joseph Garcia R.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 11.1 **Hands on Activity 11.1 Basic Algorithms Analysis**

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis (ALVAREZ)

Assess if two arrays have any common values.

```
    bool diff(int *x, int *y){
        for(int i = 0; i < y.length; i++){
            if(search(x, y[i])!= - 1){
    return false;
     }
 }
return true;
}
```

Performing worst case analysis:

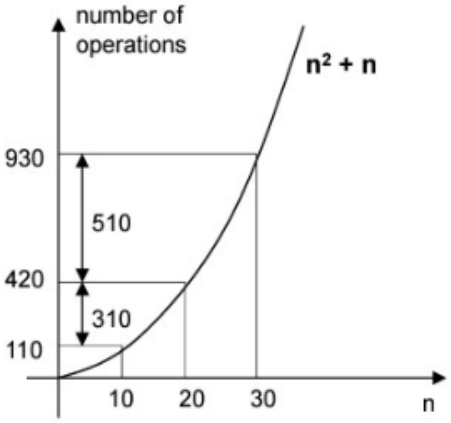
- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

TOTAL COMPARISONS = n x m
nm=n(n)= n^2

Given the above information, the total number of comparisons in the worst case is: **nm or n^2**
Worst time complexity is: **O(n^2)**.

- Assume that m = n (The arrays are the same size).

Provide an analysis of the graph for the worst case of the algorithm.

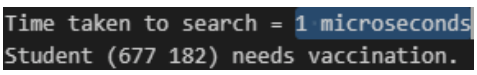
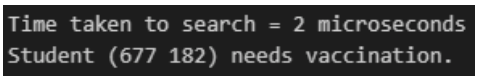
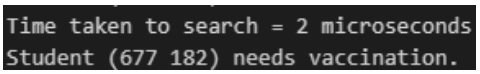


n (size of array)	number of operations
10	110
20	420
30	930

Analysis:

from what i observe, the worst case in the graph was the loop runs in n times, and each call to search requires m comparisons. Therefore, the total number of comparisons is nm. Since m=n, this becomes n^2. Including the n operations from the loop itself, so the total number of operations is n^2+n. Lastly, what I observe is that the graph shows that the number of operations increases quadratically as the array size increases. So the worst case time of complexity of the algorithm is O(n^2)

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis (GARCIA)

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1,000	1 micro-seconds		The algorithm completed the search really quick and almost instant at 1 microsecond.
10,000	2 micro-seconds		With a bigger input size, the search time or execution speed increased from 1 to 2 microseconds.
100,000	2 micro-seconds		Despite the input size multiplying by 10, which is a hundred thousand, the search time remained at 2 microseconds showing it has a high efficiency

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis **(GARCIA)**

PROBLEM 1: Consider a program that returns true if the elements in an array are unique

PSEUDOCODE

```
Algorithm Unique(A)
        for i = 0 to n-1 do
                for j = i+1 to n-1 do
                        if A[i] equals to A[j] then
                                return false
        return true
```

THEORETICAL ANALYSIS

For the worst case, the algorithm has to compare every element with every other element since there is no other duplicates to stop it early on. That means nested loops run n^2-n / 2 total comparisons, which makes O(n^2) the worst-case time complexity.

As for the best case, the very first two elements are identical, so it makes it immediately returns a “false” on the first try, giving a O(1) complexity

And lastly for the average, it checks out half the array before finding a duplicate which still comes out to O(n^2)

EXPERIMENTAL ANALYSIS

Code:

```cpp
#include <iostream>
#include <chrono>

bool is_unique(int* A, int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (A[i] == A[j]) {
                return false;
            }
        }
    }
    return true;
}

void uniqueness_test(int size) {
    int* A = new int[size];

    for (int i = 0; i < size; i++) {
        A[i] = i;
    }

    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();

    bool result = is_unique(A, size);

    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();

    std::cout << "Time taken to search = "
              << std::chrono::duration_cast<std::chrono::microseconds>(end - begin).count()
              << " microseconds" << std::endl;

    delete[] A;
}

int main() {
    uniqueness_test(1000);
    uniqueness_test(5000);
    uniqueness_test(10000);

    return 0;
}
```

Output

```
Time taken to search = 1993 microseconds
Time taken to search = 39893 microseconds
Time taken to search = 166553 microseconds
```

ANALYSIS AND COMPARISON

The results in the experimental code matches the theoretical analysis that were written. When the size of array N multiplies by 10, the number of the comparisons jumps up by 100x. The execution speed reflects this same growth, providing that the double nested loops get really slow on larger inputs or size as predicted by the worst case complexity O(n^2).

PROBLEM 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n (**ALVAREZ)**

PSEUDOCODE

```
Algorithm rpower(int x, int n):
1  if n == 0 return 1
2  else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1  if n == 0 return 1
2  if n is odd then
3       y = brpower(x, (n-1)/2)
4       return x*y*y
5  if n is even then
6       y = brpower(x, n/2)
7       return y*y
```

THEORETICAL ANALYSIS

In **rpower(),** n decreases by one during each recursion of the function; that is, n reduces to n-1, n-2, n-3, and so forth until 0. Therefore, this algorithm involves around n recursion calls, leading to a time complexity of **O(n)**.

On the other hand, in **brpower(),** n is reduced by half at each step. That means that the size of the problem reduces by half, meaning that there are fewer recursion calls, including n, n/2, n/4, n/8, and so forth. This leads to a time complexity of **O(log n)**.Considering that O(log n) increases more slowly than **O(n),** it means that **brpower()** has better time efficiency than rpower().

Time Complexity(rpower): O(n)
Space Complexity(rpower):O(n)

Time Complexity(brpower): O(log n)
Space Complexity(brpower): O(log n)

EXPERIMENTAL ANALYSIS

CODE:

HOA > HOA11p1 > Supp.cpp > main()

```cpp
#include <iostream>
#include <iomanip>
#include <chrono>

int count = 0;  // counts multiplications

double rpower(double x, int n) {
    if (n == 0) return 1;
    count++;
    return x * rpower(x, n - 1);
}

double brpower(double x, int n) {
    if (n == 0) return 1;
    if (n % 2 == 1) {
        double y = brpower(x, (n - 1) / 2);
        count += 2;
        return x * y * y;
    } else {
        double y = brpower(x, n / 2);
        count += 1;
        return y * y;
    }
}

int main() {
    int sizes[] = {1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1000};
    int runs = 10000;

    std::cout << std::left << std::setw(8) << "n"
              << std::setw(14) << "rpower_mult"
              << std::setw(14) << "brpower_mult"
              << std::setw(16) << "rpower_time(us)"
              << std::setw(16) << "brpower_time(us)" << "\n";

```

```cpp
    for (int n : sizes) {
        // count multiplications
        count = 0;
        rpower(2, n);
        int c1 = count;

        count = 0;
        brpower(2, n);
        int c2 = count;

        // measure time
        auto t1 = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < runs; i++) rpower(2, n);
        auto t2 = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < runs; i++) brpower(2, n);
        auto t3 = std::chrono::high_resolution_clock::now();

        double rTime = std::chrono::duration<double, std::micro>(t2 - t1).count() / runs;
        double bTime = std::chrono::duration<double, std::micro>(t3 - t2).count() / runs;

        std::cout << std::left << std::setw(8) << n
                  << std::setw(14) << c1
                  << std::setw(14) << c2
                  << std::setw(16) << std::fixed << std::setprecision(4) << rTime
                  << std::setw(16) << bTime << "\n";
    }
    return 0;
}
```

OUTPUT

```
[Running] cd "c:\Users\RYZEN 5\OneDrive\DSA\HOA\HOA11p1\" && g++ Supp.
cpp -o Supp && "c:\Users\RYZEN 5\OneDrive\DSA\HOA\HOA11p1\"Supp
n       rpower_mult   brpower_mult  rpower_time(us) brpower_time(us)
1       1             2             0.0031          0.0042
2       2             3             0.0043          0.0069
4       4             4             0.0083          0.0085
8       8             5             0.0153          0.0107
16      16            6             0.0300          0.0132
32      32            7             0.0677          0.0252
64      64            8             0.1391          0.0201
128     128           9             0.2622          0.0232
256     256           10            0.5222          0.0383
512     512           11            1.0730          0.0298
1000    1000          16            2.0759          0.0343
```

RESULTS IN MULTIPLICATION COUNTS

n	rpower_mult	brpower_mult	rpower_time(us)	brpower_time(us)
1	1	2	0.0031	0.0042
2	2	3	0.0043	0.0069
4	4	4	0.0083	0.0085
8	8	5	0.0153	0.0107
16	16	6	0.0300	0.0132
32	32	7	0.0677	0.0252
64	64	8	0.1391	0.0201
128	128	9	0.2622	0.0232
256	256	10	0.5222	0.0383
512	512	11	1.0730	0.0298
1000	1000	16	2.0759	0.0343

- In this table that I observed was that each time n doubles, rpower's count doubles which defines as (linear growth).
- In opposite of rpower which is brpower, each time n doubles the brpower count increases by only 1 which defines as (logarithmic growth).
- For n = 1000, brpower needs 6x2 +4x1 = 16 multiplications, while rpower is 1000.

-

ANALYSIS AND COMPARISON

Type	rpower	brpower
Recurrence	T(n) = T(n-1) + 1	T(n) = T(n/2) + c
Time complexity	Θ(n)	Θ(log n)
Space(Stack)	Θ(n)	Θ(log n)
Reduces n by	1 per call	half per call
N = 1000	1000 mults	16 mults

In the case of rpower(), the number of multiplications required is always n (for n=8, the number of multiplications is 8; for n=64, it is 64; for n=1000, it is 1000). It thus increases linearly as O(n). In case of brpower(), the number of multiplications increases by just 1 at every doubling of n (for n=8, it is 5; for n=16, it is 6; for n=32, it is 7). Thus, its complexity is logarithmic in nature, which is represented as O(log n).

In the case of very small n (that is, n=1 and n=2), brpower() performs slightly more multiplication operations than rpower() (that is, 2 as compared to 1, and 3 as compared to 2), as the function performs one or two multiplication operations in every call. However, it becomes insignificant as n grows larger. While rpower() requires 1000 multiplications when n=1000, brpower() requires only 16 multiplications.

Conclusion	
Provide the following: Summary of lessons learned Analysis of the procedure Analysis of the supplementary activity Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?	
Garcia	In this activity I learned how to analyze the efficiency of algorithms by using both theoretical operation counting and experimental timing using the **chrono** standard library. In the procedure, **diff** runs in **O(n^2)** worst case time, while the binary search in the student vaccination program was faster because it splits the dataset in half each time. For the Supplementary activity, Unique(A) showed **O(n^2)** worst case and **O(1)** best case short circuiting, **brpower** demonstrated an efficient **O(log n)** approach compared to **rpowers O(n)**, and the measured execution times followed these theoretical curves close. I think i performed well in this activity since my experimental results backed up the theory, but i think i still need to work and improve at isolating the background system overhead during microsecond level tests.
Alvarez	From this activity, I was able to find two approaches of finding out the efficiency of an algorithm. These two approaches were determining the **time complexity** of an algorithm from scratch and testing the algorithm in a program. After the test using the number of operations and the time function from the **chrono library**, I realized that there is agreement between the **Big-O** values and the results. In the procedure section, I filled in the blank and described the program from the given code. I found out that **diff()** is a **quadratic function** because for every one of the n items, **search()** is done up to m times. In the experiment, the time taken by the search algorithm remained almost the same **(about 1 to 2 microseconds for 100,000 inputs)** because of the fast growth of the **binary search.** In the supplementary activity, I understood that the number of times a function calls itself defines its complexity. For example, **rpower()** makes **n calls** while **brpower()** makes **log n** calls because the problem is divided by 2 at every iteration. In my opinion, I understood how to get **Big-O** from the code but not how to construct it while coding. .

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly, Chatgpt,claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, explanation of concepts.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of

concepts, etc.

Extent of Use: **Ai was used to help write and debug the C++ code, and help to analyze the algorithms and suggest what is wrong, We ran the code ourselves, reviewed and understand each of the code and filled in the results from our own program output, The Ai assisted our work but did not replace our own understanding, analysis or completion of the required activity**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.